

PROLOG



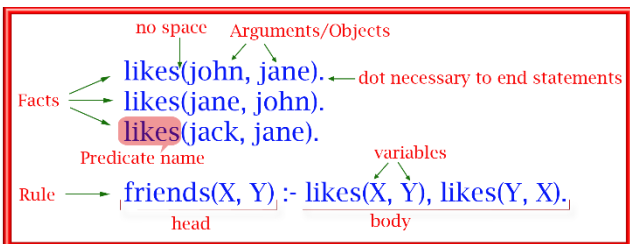
Prolog, mantıksal analizlerde kullanılan yüksek seviyeli bir programlama dilidir. Prolog dili, 1970'li yıllarda Alain Colmerauer tarafından tasarlanmıştır. İlk mantık programlama dillerinden biri olma ünvanına sahiptir ve günümüzde sıklıkla kullanılmaktadır. Bir mantıksal programlama (logic programming) dilidir. İsmi "Programming in Logic" ifadesinden gelir.

Prolog dilinin başlıca kullanım alanı Yapay Zeka operasyonlarıdır. İfadeler mantığını kullanarak, bilgisayara çözüm aranan problem ile ilgili gerçek ve kuralları vererek en uygun bir çözümün elde edilmesini sağlar.

Prolog Dilinin Temel Yapısı

Prolog'u diğer dillerden ayıran en temel özellik Deklaratif (Bildirimsel) bir paradigma kullanmasıdır. Prolog dili, bilinen diğer programlama dilleri (C, JavaScript, Python) gibi komut satırlarından değil, bilinirliği önceden belirlenmiş gerçeklerden (Facts) ve bu gerçekler kullanılarak bilgi sağlayan bir takım kurallardan (Rules) oluşmaktadır. Bu yapılara ise Clause adı verilmektedir. Yani çoğu programlama dilinde bilgisayara bir şeyin nasıl yapılacağını anlatırken Prolog'da ise genellikle neyin doğru olduğunu tanımlarız. Bilgisayar da sonuca çıkarım motoru (Inference engine) sayesinde kendisi ulaşır.

Prolog, Matematiksel Mantık (First-Order Predicate Calculus) üzerine kuruludur. Program bir algoritma değil, bir veri tabanı (knowledge base) gibidir.



Sözdizimi (Syntax) ve Anlam (Semantics) Yapısı

Prolog syntax'ı oldukça sade ama katıdır. Üç temel yapı vardır:

1- Gerçekler (Facts): Kesin doğru olduğu bilinen ifade ve bilgiler. Küçük harfle başlar.

`baba(ahmet, mehmet).`

(Ahmet, Mehmet'in babasıdır.)

2- Kurallar (Rules): Yeni bilgiler türetmek için kullanılır. "Eğer" (:-) koşulu.

`dede(X, Z) :- baba(X, Y), baba(Y, Z).`

(X, Z'nin dedesidir EĞER X, Y'nin babasıysa VE Y, Z'nin babasıysa.)

3- Sorgular (Queries): Program çalışırken sisteme sorulan sorular.

`?- dede(ahmet, Torun).`

(Ahmet kimlerin dedesidir?)

Semantik (Anlam): Prolog yukarıdan aşağıya ve soldan sağa doğru çalışır. Bir çözümü ararken *Unification (Eşleme)* ve *Backtracking (Geriye Dönme)* mekanizmalarını kullanır. Eğer bir yol çıkmaza girerse, otomatik olarak bir önceki adıma dönüp alternatif yolları dener.

Unification (Birleştirme): Örneğin:

`seviyor(ali, pizza).`

`X = ali`

`?- seviyor(X, pizza).`

Sonucunu üretir.

Backtracking (Geri İzleme): Prolog başarısız olan yolları bırakıp başka olasılıkları otomatik dener. Buna backtracking denir.

Nesneye Yönelik Kavramlar (OOP)

Prolog saf halinde nesne yönelimli bir dil değildir; Sınıf (Class), Nesne (Object) veya Kalıtım (Inheritance) gibi yapılar barındırmaz. Nesne yönelimli mantık, "Gerçekler" ve "Kurallar" üzerinden simüle edilir. Kalıtım (Inheritance) ilişkisi kurallarla yazılır:

hayvan(kedi).

memeli(X) :- kedi(X). % Kedi olan her şey memelidir (Kalıtım)

Yani dilin doğası gereği OOP yoktur, ama mantıksal kurallarla nesne ilişkileri modellenir.

Fonksiyonel Yapılar

Prolog'da teknik olarak "fonksiyon" yoktur, Predikatlar (Bağıntılar/Yüklemeler) vardır. Fonksiyonlar bir değer döndürür ($f(x) = y$), Prolog predikatları ise sadece doğru (true) veya yanlış (false) döndürür.

Fonksiyonel dillerin vazgeçilmezi olan Recursion (Özyineleme) ve List Processing (Liste İşleme) Prolog'da oldukça güçlüdür.

Faktoriyel hesaplama örneği:

```
faktoriyel(0, 1). % Base case (Taban durum)
faktoriyel(N, F) :-
    N > 0,
    N1 is N - 1,
    faktoriyel(N1, F1),
    F is N * F1.
```

Tipler, ADT ve Genericlik

Prolog dinamik olarak tiplendirilmiş (dynamically typed) bir dildir. Yani değişkenlerin tipini önceden tanımlamazsınız.

Veri Tipleri: Temel olarak tek bir veri tipi vardır: Term (Terim). Terimler kendi içinde ayrılır:

Atoms: Genellikle büyük veya küçük harflerden, rakamlardan oluşan kelime parçalarıdır. Sabit değerlerdir. Küçük harflerle başlarlar. *Örn: elephant, b, abcXYZ, x_123, 'This is also a Prolog atom.'*

Numbers: Tam sayılar ve ondalıklı sayıları temsil etmektedir.

Variables: Değişkenler olarak ifade edilebilir. Büyük harfle başlarlar. *Örn: X, Elephant, _4711, X_1_2, MyVariable*

Compound Terms: Bir Prolog atomundan oluşmaktadır ve parantez içerisine alınan, virgülle ayrılan bir dizi argümanlar olarak tanımlanabilir. *Örn: tarih(5, haziran, 2026)*

ADT (Soyut Veri Tipleri): Listeler ([Baş | Kuyruk]) ve Ağaç (Tree) yapıları compound term'ler kullanılarak kolayca oluşturulur.

Genericlik: Prolog doğuştan generic'tir. Tip tanımı olmadığı için yazılan bir listeyi tersine çevirme (reverse) kuralı, listenin içinde sayı da olsa, atom da olsa, başka listeler de olsa aynı şekilde çalışır.

Binding (Bağlama) ve Bellek Yönetimi

Binding: Prolog'da değişken bağlama işlemi "Logical Binding" (Mantıksal Bağlama) ile yapılır ve bu işlem tek seferliktir (single assignment). Bir X değişkeni bir değere eşitlendiğinde (Unification), backtracking olana kadar o değer değiştirilemez. Yani Imperatif dillerdeki gibi $X = X + 1$ diyemezsiniz.

Bellek Yönetimi: Prolog bellek yönetimi için Stack ve Heap kullanır. Otomatik bir Garbage Collector (Çöp toplayıcı) mekanizmasına sahiptir. Backtracking gerçekleştiğinde, başarısız yollardaki değişken bağlamaları bellekten otomatik olarak temizlenir.

Paralellik ve Uygulanması

Prolog mantıksal yapısı gereği paralellığe (Implicit Parallelism) çok yatkındır. İki tür paralellik vardır:

1- OR-Parallelism: Bir sorgunun eşleşebileceği birden fazla kural varsa, sistem bu kuralları aynı anda farklı işlemcilerde arayabilir. *Örn: baba(ahmet, X) sorgusunda tüm Ahmet'leri aynı anda aramak*

2- AND-Parallelism: Bir kuralın sağ tarafındaki birden fazla koşulun aynı anda çalıştırılmasıdır. *Örn:*

dede(X,Z) :- baba(X,Y), baba(Y,Z). ifadesinde iki baba kontrolünün paralel yapılması gibi.

Modern Prolog derleyicileri (Örn: SICStus Prolog veya bazı açık kaynaklı paralel kütüphaneler) bu süreçleri multithreading ile yönetir.

Atama

Prolog'da bildiğimiz anlamda bir atama (=) operatörü yoktur. = operatörü Prolog'da bir atama değil, Unification (Eşleme) operatörüdür (X = 5 demek, "X'i 5 ile eşle" demektir). Matematiksel bir ifadeyi hesaplayıp sonucunu bir değişkene bağlamak için is operatörü kullanılır:

Sayi is 3 + 4. % Sayi değişkeni 7 olur.

Quality Attributes

- Ortogonallik: Prolog'da ortogonallik oldukça yüksektir. Dilde neredeyse her şey tek bir kavram üzerine kuruludur.
- Prolog'da "Boolean" veri tipi yoktur. Bir ifadenin true veya false dönmesi, imperatif dillerdeki gibi bir veri hücresinin değeri değil; kurulan mantıksal sorgunun veri tabanında başarıya (success) ulaşmış olup olmadığına ya da başarısız (failure) olduğunun bir sonucudur.
- Söz dizimi tasarımı son derece minimalisttir. Okunabilirliği ilk başta zordur çünkü alışlagelmiş algoritma kalıplarını (döngüler, if-else blokları) barındırmaz.
- Yapay zeka, grafik arama ve kısıt çözme (constraint solving) gibi alanlarda ifade edilebilirliği inanılmaz yüksektir. Diğer dillerde 100 satır sürececek bir derinlik

öncelikli arama algoritması, Prolog'da sadece 3-4 satırda ifade edilebilir. Ancak genel amaçlı işlerde (örneğin dosya G/Ç işlemleri) yazılabilirliği düşüktür.

- Güvenilirlik (Reliability): Prolog'da yan etkiler (side-effects) neredeyse yoktur ve değişken atamaları tek seferliktir (single assignment). Bu durum kodun güvenilirliğini artırır. En büyük güvenlik riski ise mantık hataları yüzünden programın sonsuz döngüye girmesidir.
- Maliyet (Cost): Derleyicileri oldukça hafiftir ve geliştirme maliyeti düşüktür. Ancak çalışma zamanında (runtime) sürekli geriye dönme (backtracking) yaptığı için işlemci ve bellek maliyeti yüksek olabilir.

Programın Çalıştırılması

Modern Prolog sistemleri (en popüler olan SWI-Prolog gibi) Hibrit Yaklaşım kullanır. Kod önce WAM (Warren Abstract Machine) adı verilen, Prolog için optimize edilmiş bir sanal makinenin anlayacağı ara koda (bytecode) derlenir. Ardından bu bytecode bir sanal makine üzerinde yorumlanır (interpreted). Bazı gelişmiş ticari derleyiciler ise performansı artırmak için yerel koda (native code) derleme yapar.

Sunum Video Linki:

<https://www.youtube.com/watch?v=8MTdLIKgaTE>

Kaynakça:

- <https://builtin.com/software-engineering-perspectives/prolog>
- <https://medium.com/t%C3%BCrk-telekom-bulut-teknolojileri/prolog-programlama-diline-giri%C5%9F-896a774a54aa>
- <https://athena.ecs.csus.edu/~mei/logic/p/prolog/programming-examples.html>